



# Pontificia Universidad Javeriana

## Departamento de Ingeniería de Sistemas

### Estructuras de Datos

#### Taller 2: Estructuras Lineales, 2026-10

## 1. Objetivo

Completar un simulador de batallas Pokémon mediante operaciones de búsqueda, inserción, reorganización, recorrido y análisis sobre estructuras lineales de la STL.

## 2. Recordatorio: compilación con g++

La compilación con g++ (compilador estándar que será usado en este curso para evaluar y calificar las entregas) se realiza con los siguientes pasos:

1. **Compilación:** de todo el código fuente compilable (**ÚNICAMENTE LOS ARCHIVOS CON EXTENSIONES** \*.c, \*.cpp, \*.cxx)

```
g++ -std=c++17 -c *.c *.cxx *.cpp
```

2. **Encadenamiento:** de todo el código de bajo nivel en el archivo ejecutable

```
g++ -std=c++17 -o nombre_de_mi_programa *.o
```

Nota: Estos dos pasos (compilación y encadenamiento) pueden abreviarse en un sólo comando:

```
g++ -std=c++17 -o nombre_de_mi_programa *.c *.cxx *.cpp
```

3. **Ejecución:** del programa ejecutable anteriormente generado

```
./nombre_de_mi_programa
```

**ATENCIÓN:** Los archivos de encabezados (\*.h, \*.hpp, \*.hxx) **NO SE COMPILAN**, se incluyen en otros archivos (encabezados o código). Así mismo, los archivos de código fuente (\*.c, \*.cpp, \*.cxx) **NO SE INCLUYEN**, se compilan. Si el programa entregado como respuesta a este Taller no atiende estas recomendaciones, automáticamente se calificará la entrega sobre un 25 % menos de la calificación máxima.

## 3. Desarrollo del taller

Pokémon es una franquicia centrada en criaturas que pueden ser entrenadas y organizadas en equipos para participar en enfrentamientos. Cada Pokémon posee características particulares que determinan su desempeño durante una batalla, entre ellas su tipo elemental, salud, capacidad de ataque, defensa y velocidad.

En los videojuegos originales intervienen numerosas reglas, movimientos, estados y características especiales. Para este taller se utilizará un modelo deliberadamente simplificado. El objetivo no es reproducir completamente un videojuego, sino representar un pequeño sistema de batallas que permita estudiar la organización y manipulación de información mediante estructuras lineales.

La aplicación suministrada permite registrar información relacionada con Pokémon, equipos y entrenadores, así como simular enfrentamientos y consultar sus resultados. El sistema también conserva información sobre las batallas realizadas y permite efectuar comparaciones entre los participantes.

El proyecto está construido a partir de varios Tipos Abstractos de Datos que colaboran entre sí. Cada uno representa una parte del dominio y administra su información mediante diferentes estructuras lineales de la biblioteca estándar de C++.

A lo largo del taller, el estudiante deberá estudiar el código suministrado para comprender:

- La información representada por cada TAD.
- Las relaciones existentes entre los diferentes TAD.
- Las estructuras lineales utilizadas.

- Las reglas que controlan el funcionamiento del simulador.
- Los invariantes que deben conservarse.
- El recorrido y modificación de las secuencias almacenadas.

El desarrollo consistirá en completar los elementos identificados con comentarios **TODO**. Las implementaciones deberán respetar las interfaces existentes e integrarse correctamente con el resto de la aplicación.

No se deben eliminar los comentarios **TODO** ni modificar las firmas de las funciones, salvo que el enunciado lo autorice expresamente. Al finalizar, el programa deberá compilar sin errores ni advertencias y comportarse correctamente ante los casos de prueba proporcionados.

1. **(10 %) Investigación sobre el sistema de batallas Pokémon** Antes de analizar e implementar el código, realice una investigación sobre el funcionamiento general de los equipos y las batallas en los videojuegos de Pokémon. El propósito de esta investigación no es estudiar todas las reglas existentes ni desarrollar un simulador completo. Se busca comprender los conceptos fundamentales del dominio representado por la aplicación y reconocer las decisiones de simplificación necesarias para convertirlo en un problema manejable dentro del curso. Oriente su investigación mediante las siguientes preguntas:

- ¿Qué es un Pokémon y cuáles son sus características principales?
- ¿Cómo se conforma un equipo Pokémon?
- ¿Existe un límite para la cantidad de integrantes de un equipo?
- ¿Por qué es importante el orden de los Pokémon dentro de un equipo?
- ¿Qué estadísticas pueden influir en el desarrollo de una batalla?
- ¿Qué representan los puntos de salud?
- ¿Cuándo se considera que un Pokémon está debilitado?
- ¿Cómo se determina qué Pokémon actúa primero?
- ¿Qué es un ataque y qué elementos pueden influir en el daño producido?
- ¿Qué son los tipos elementales?
- ¿Cómo puede el tipo del atacante afectar a un oponente?
- ¿Qué significa que un ataque sea efectivo, poco efectivo o neutral?
- ¿Qué sucede cuando el Pokémon activo queda debilitado?
- ¿En qué condiciones termina normalmente una batalla?
- ¿Cómo podría determinarse el ganador de un enfrentamiento?
- ¿Qué información resultaría relevante conservar en un historial de batalla?

En el reporte, presente una explicación organizada de los conceptos encontrados. No responda las preguntas como una lista aislada; utilícelas como guía para construir un texto que permita comprender el funcionamiento general de un equipo y una batalla Pokémon.

No se espera una descripción exhaustiva de todas las generaciones, versiones o modalidades de los videojuegos. La investigación debe concentrarse en los elementos que resulten pertinentes para comprender el problema representado por la aplicación.

La evaluación considerará la calidad de las fuentes, la claridad de la explicación, la correcta diferenciación entre el sistema original y el simulador académico, y la capacidad para relacionar la investigación con el código suministrado.

### **Simplificación del dominio**

Los videojuegos de Pokémon incluyen una cantidad considerable de reglas, entre ellas movimientos con características particulares, precisión, efectos secundarios, estados alterados, habilidades, objetos, niveles, cambios de estadísticas y diferentes modalidades de combate.

El simulador utilizado en este taller no pretende reproducir toda esa complejidad. Algunas reglas fueron seleccionadas, otras fueron simplificadas y algunas fueron adaptadas con fines académicos para facilitar su representación mediante Tipos Abstractos de Datos y estructuras lineales.

2. **(5 %) Análisis del procedimiento principal** Revise completamente el archivo main.cxx y analice el procedimiento seguido por la aplicación desde su inicio hasta su finalización. En el reporte, construya un diagrama de flujo que represente el comportamiento general del programa. El diagrama debe incluir: Inicio y finalización. Lectura de información. Validaciones principales. Procesamiento de operaciones. Decisiones que pueden producir errores. Presentación de resultados.

No es necesario representar cada instrucción del código. El diagrama debe mostrar el flujo lógico de la aplicación y la relación entre sus procedimientos principales.

Además, identifique todas las funciones definidas en main.cxx y complete la siguiente tabla:

| Función              | Propósito         | Datos de entrada                   | Resultado                          | Funciones que utiliza           |
|----------------------|-------------------|------------------------------------|------------------------------------|---------------------------------|
| Nombre de la función | Descripción breve | Parámetros o información procesada | Valor retornado o efecto producido | Funciones relevantes que invoca |

Cada descripción deberá:

- Tener un máximo de 20 palabras.
- Expresar el propósito de la función, no repetir literalmente su nombre.
- Diferenciar entre valores retornados y efectos producidos.
- Indicar cuándo una función modifica el estado del simulador.
- Mencionar los principales errores que puede detectar.

**No copie el código fuente en el reporte.** El propósito de este punto es demostrar que se comprende el flujo de ejecución y la responsabilidad de cada función.

3. **(5 %) Corrección de errores y advertencias** Intente compilar el código. En el reporte, haga un inventario conciso de los errores y advertencias que aparecen (**NO copiar y pegar tal cual**), y proponga formas de solucionarlos. Aplique estas soluciones en el código fuente. Después de esto, el código debería compilar sin errores ni advertencias.

El programa contiene errores introducidos intencionalmente. Algunos impiden generar el archivo ejecutable, mientras que otros solamente podrán identificarse después de compilar y ejecutar la aplicación.

Tenga en cuenta que un programa puede compilar correctamente y aun presentar errores lógicos. Por tanto, obtener el archivo ejecutable no será suficiente para completar este punto.

La información debe presentarse en una tabla organizada, con las siguientes columnas:

| Tipo de error           | Descripción del error                                                                                    | Causa probable del error                                                                                                                                                               | Solución aplicada                                                                                                   | Resultado tras corrección                                            |
|-------------------------|----------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------|
| Compilación / Ejecución | Indique brevemente el tipo de error que se produjo (sin copiar el mensaje del compilador o la terminal). | Explique por qué ocurrió el error (por ejemplo: variable no declarada, acceso a índice inválido, etc.). Incluya una imagen que muestre el trozo de código donde se encuentra el error. | Describa qué cambio realizó en el código para solucionarlo. Incluya una imagen que muestre la corrección realizada. | Indique si el problema se resolvió o si surgió otro error posterior. |

Cada fila debe estar redactada en lenguaje técnico, claro y conciso. El propósito de este ejercicio es desarrollar la habilidad de interpretar y resolver problemas de compilación y ejecución de forma argumentada y profesional.

4. (10 %) **Análisis y especificación de los Tipos Abstractos de Datos** Antes de continuar con la implementación, revise completamente todos los archivos de encabezado .h y de código fuente .cxx suministrados. Durante esta revisión, determine si comprende:

- El propósito general de cada archivo.
- Los atributos declarados en las clases.
- Las operaciones públicas y privadas.
- Los parámetros y tipos de retorno.
- El uso de referencias constantes y modificables.
- Las estructuras lineales empleadas.
- El funcionamiento de los iteradores.
- Las relaciones entre las clases.
- Las precondiciones y poscondiciones documentadas.
- Los valores utilizados para representar búsquedas fallidas.
- La manera en que los objetos son creados, almacenados y modificados.

Después de revisar el código, responda en el reporte: **¿Comprende completamente todos los elementos presentes en los archivos .h y .cxx del proyecto?**

La respuesta no debe limitarse a “sí” o “no”. Si encontró conceptos, instrucciones o decisiones de diseño que inicialmente no comprendía, complete la siguiente tabla:

| Elemento consultado                 | Archivo donde aparece | Duda inicial      | Fuente consultada    | Explicación obtenida             |
|-------------------------------------|-----------------------|-------------------|----------------------|----------------------------------|
| <b>Concepto, tipo o instrucción</b> | Nombre del archivo    | Qué no comprendía | Referencia utilizada | Explicación con palabras propias |

Puede utilizar documentación técnica, libros, material del curso o consultas realizadas al profesor y los monitores. Las fuentes externas deben citarse correctamente.

A partir del código analizado, identifique los Tipos Abstractos de Datos que conforman la aplicación. Para

cada TAD identificado, presente la especificación detallada. Utilice el formato visto en clase como referencia:

- Nombre del TAD
- Descripción informal del propósito del TAD
- Estado interno (atributos que lo definen)
- Interfaz (lista de operaciones públicas con su tipo de entrada y salida, sin implementación)

Además, se debe incluir un diagrama de relación entre los TADs, en el que se evidencie la forma en que se componen o interactúan entre sí. La especificación debe reflejar fielmente lo que está implementado en el código. No se aceptarán definiciones genéricas o copiadas de internet.

Este taller se enfoca en el desarrollo de una aplicación para administrar entrenadores, organizar equipos y simular batallas Pokémon mediante el uso de estructuras lineales. Para completar el programa, se deberán implementar de manera organizada y metódica todos los elementos identificados con comentarios "TODO".

**Favor de no borrar el comentario "TODO" para facilitar el proceso calificativo.**

**El código desarrollado debe ser incluido dentro del informe como una imagen adjunta**

5. **(5 %) TODO #01: Búsqueda modificable de un Pokémon:** Implemente la operación que busca un Pokémon por su nombre dentro del equipo y retorna un iterador modificable a la posición donde se encuentra. Este iterador permitirá que otras operaciones consulten o modifiquen directamente el Pokémon almacenado. Si no existe un Pokémon con el nombre indicado, la función deberá retornar la posición definida por el TAD para representar una búsqueda fallida. Consulte los comentarios incluidos en el código para conocer las condiciones específicas del recorrido y del valor retornado.
6. **(5 %) TODO #02: Búsqueda constante de un Pokémon:** Implemente la versión de la operación de búsqueda que puede ejecutarse sobre un equipo constante y que retorna un iterador de solo consulta. Compare esta función con la desarrollada en el TODO anterior e identifique por qué ambas pueden tener el mismo nombre y parámetros, pero retornar iteradores con diferentes permisos de acceso. Si el Pokémon no existe, deberá retornarse la posición constante definida por el TAD para representar una búsqueda fallida. Consulte los comentarios del código para conocer las condiciones específicas de implementación.
7. **(5 %) TODO #03: Inserción de un Pokémon en el equipo:** Implemente la operación encargada de agregar un Pokémon en una posición específica de la secuencia. Antes de realizar la inserción, deberán validarse las restricciones definidas por el TAD, incluyendo la capacidad máxima del equipo, la ausencia de nombres repetidos y la validez de la posición solicitada. La implementación debe utilizar las operaciones de búsqueda desarrolladas anteriormente y conservar el orden relativo de los Pokémon que ya se encuentran almacenados. Consulte los comentarios del código para conocer los casos de rechazo y el procedimiento esperado para localizar la posición de inserción.
8. **(10 %) TODO #04: Determinación de la derrota de un equipo:** Implemente la operación que determina si un equipo ya no cuenta con Pokémon disponibles para continuar una batalla. La solución deberá reutilizar las operaciones de búsqueda suministradas por el TAD, evitando repetir innecesariamente el recorrido de la estructura. Considere tanto los equipos cuyos integrantes están completamente debilitados como aquellos que no contienen Pokémon. Consulte los comentarios del código para identificar las operaciones que deben utilizarse y la condición exacta del resultado.
9. **(10 %) TODO #05: Análisis de la efectividad entre tipos:** Estudie la operación que determina el multiplicador aplicado según los tipos de los Pokémon involucrados en un ataque. Esta función se entrega implementada y no debe modificarse. A partir de su análisis, construya en el reporte un diagrama dirigido que represente las relaciones de efectividad definidas por el simulador y explique cómo estas relaciones afectan el cálculo del daño. También deberá considerar el carácter direccional de la comparación y el comportamiento de las combinaciones que no aparecen explícitamente en las condiciones. Consulte los comentarios del código para conocer los elementos que debe contener el análisis.
10. **(10 %) TODO #06: Ejecución de un ataque:** Implemente la operación encargada de ejecutar el ataque de un Pokémon contra otro y registrar sus efectos en el historial de la batalla. La función deberá utilizar el cálculo

de daño suministrado, modificar correctamente la salud del defensor y conservar una descripción cronológica de lo sucedido. También deberá reconocer cuándo el ataque produce el debilitamiento del oponente y registrar este acontecimiento de manera independiente. Consulte los comentarios del código para conocer el orden de las operaciones y la información que debe almacenarse en cada evento.

- 11.(5 %) **TODO #07: Ejecución completa de una batalla:** Implemente la operación principal que controla un enfrentamiento entre dos equipos Pokémon. La función deberá coordinar las operaciones desarrolladas y suministradas previamente para seleccionar los participantes, determinar el orden de ataque, ejecutar los turnos, reemplazar a los Pokémon debilitados y reconocer el final del enfrentamiento. También deberá manejar adecuadamente los equipos que no tengan Pokémon disponibles desde el inicio, conservar el historial cronológico y registrar tanto el equipo como el entrenador ganador. Consulte los comentarios del código para seguir las etapas, condiciones especiales y reglas de terminación establecidas para el simulador.
- 12.(10 %) **TODO #08: Ejecución de una batalla seleccionada:** Implemente la operación del simulador que localiza dos entrenadores y sus respectivos equipos a partir de los nombres recibidos, valida su existencia y coordina la ejecución de una batalla entre ellos. Cuando todos los participantes sean válidos, la batalla deberá construirse con la información correspondiente, ejecutarse completamente y almacenarse en el historial general del simulador. La operación deberá rechazar cualquier solicitud que contenga un entrenador o equipo inexistente, sin modificar el historial. Consulte los comentarios del código para conocer el orden de las búsquedas, la construcción del enfrentamiento y las condiciones de retorno.
- 13.(10 %) **TODO #09: Comparación completa de entrenadores:** Implemente la operación que compara dos entrenadores enfrentando cada equipo del primero contra todos los equipos del segundo. La solución deberá validar los participantes, construir los recorridos anidados necesarios, ejecutar y conservar cada batalla, registrar el resultado individual de ambos equipos y calcular sus porcentajes de victoria. Las estadísticas de la comparación más reciente deberán actualizarse sin eliminar el historial general de enfrentamientos. Consulte los comentarios del código para conocer las validaciones, operaciones auxiliares y etapas necesarias para completar el proceso.

Una vez implementados los puntos anteriores, su código debería poder ejecutarse sin problemas. Sin embargo, el hecho de que el código se ejecute correctamente no garantiza que cumpla con todos los requisitos de correctitud. Para asegurar su precisión y conformidad con las especificaciones, en los siguientes puntos se desarrollará un protocolo de pruebas que validará su correcto desempeño.

- 14.(10 %) **Caso de prueba básico.** Revise cuidadosamente los archivos `trainers.txt` y `operations.txt` suministrados con el proyecto. A partir de la información contenida en ellos, determine qué entrenadores, equipos, Pokémon y operaciones participarán durante la ejecución. Antes de ejecutar el programa, responda:

- ¿Cuántos entrenadores se encuentran registrados?
- ¿Cuántos equipos tiene cada entrenador?
- ¿Qué operaciones serán ejecutadas y en qué orden?
- ¿Qué equipos participarán en la batalla individual?
- ¿Qué Pokémon debería atacar primero en cada enfrentamiento? Justifique usando su velocidad.
- ¿Cómo influyen los tipos de los Pokémon en el daño?
- ¿Qué entrenador y equipo espera que ganen la batalla individual?
- ¿Cuántas batallas debería generar la comparación de entrenadores?
- ¿Cuántas batallas deberían aparecer en el historial al finalizar?
- ¿Cuáles equipos espera que obtengan los mejores resultados?

Después, ejecute el programa y compare sus respuestas con los resultados obtenidos. Registre la validación en la siguiente tabla:

| <i>Elemento analizado</i>        | <b>Resultado esperado</b> | <b>Resultado obtenido</b> | <b>¿Coincide?</b> | <b>Explicación</b> |
|----------------------------------|---------------------------|---------------------------|-------------------|--------------------|
| <i>Batalla individual</i>        |                           |                           |                   |                    |
| <i>Ganador</i>                   |                           |                           |                   |                    |
| <i>Cantidad de batallas</i>      |                           |                           |                   |                    |
| <i>Resultados de comparación</i> |                           |                           |                   |                    |
| <i>Mejores equipos</i>           |                           |                           |                   |                    |

Incluya evidencia visual de la ejecución y redacte una conclusión de máximo 150 palabras. Si encuentra diferencias, indique si se originan en su análisis previo, en los datos de entrada o en la implementación del programa.

## 4. Evaluación

La entrega se hará a través de la correspondiente actividad de Brightspace, antes de la media noche del **domingo 25 de agosto de 2026**. Se debe entregar el **documento reporte (.pdf)** y un **archivo comprimido** (únicos formatos aceptados: .zip, .tar, .tar.gz, .tar.bz2, .tgz) que contenga dentro de un mismo directorio (sin estructura de carpetas interna) el código fuente modificado (.h, .hxx, .hpp, .c, .cxx, .cpp).

Si la entrega contiene archivos en cualquier otro formato o más archivos de los solicitados en esta descripción, será descartada y no será evaluada, es decir, la nota definitiva de la entrega será de 0 (cero) sobre 5 (cinco).

Si el archivo comprimido no cuenta con las especificaciones descritas (*sin estructura de carpetas interna*), la nota definitiva de la entrega será de 0 (cero) sobre 5 (cinco).

Si el código fuente no compila, la nota definitiva de la entrega será de 0 (cero) sobre 5 (cinco).

(60%) **Resultado del trabajo en grupo (informe):** el resultado consignado en el informe se evaluará con la siguiente escala para cada uno de los puntos:

La evaluación del taller tendrá la siguiente escala para cada una de las preguntas o secciones de código a completar:

- **Excelente (5.0/5.0):** El código es correcto, compila sin advertencias y/o la respuesta es correcta y concisa.
- **Bueno (3.5/5.0):** El código es correcto, pero compila con advertencias y/o la respuesta es correcta pero no es concisa.
- **No fue un trabajo formal de ingeniería (3.0/5.0):** El código es parcialmente correcto y/o compila con advertencias y/o la respuesta es aproximada y/o no es concisa.
- **Necesita mejoras sustanciales (2.0/5.0):** El código no es correcto y/o la respuesta no es clara.
- **Malo (1.0/5.0):** El código entregado por el estudiante no compila en el compilador g++
- **No entregó (0.0/5.0).**

Para el caso de las secciones correspondientes al reporte se manejará la siguiente escala:

- **Excelente (5.0/5.0):** El estudiante presenta análisis y explicaciones organizados (información) que permiten comprender en su totalidad los procedimientos, funciones o errores.
- **Bueno (4.0/5.0):** El estudiante tiene una idea parcial de los procedimientos, funciones o errores, pero falla en el formalismo de la presentación escrita. No presenta las secciones escritas de manera organizada lo que dificulta la comprensión de las ideas.
- **Regular (3.0/5.0):** El estudiante logra esbozar una idea de los procedimientos, funciones o errores, falla en presentarla de manera clara y contundente. No presenta claridad en sus explicaciones.
- **Malo (1.5/5.0):** El estudiante no logra entender las instrucciones presentadas, y su análisis es vago o ambiguo.

- **No entregó (0.0/5.0):** No entregó el archivo del informe, o el archivo entregado no está en formato PDF.

**(40%) Evaluación individual del resultado del taller:** cada estudiante, de forma individual, deberá responder algunas preguntas sobre el trabajo realizado. Cada pregunta se evaluará así:

- **Excelente (5.0/5.0):** La respuesta es correcta y concuerda totalmente con la entrega realizada por el grupo de trabajo.
- **Bueno (3.0/5.0):** La respuesta es parcialmente correcta y/o concuerda sólo en algunos aspectos con la entrega realizada por el grupo de trabajo.
- **Malo (1.0/5.0):** La respuesta no es correcta y/o no concuerda de ninguna forma con la entrega realizada por el grupo de trabajo.
- **No entregó (0.0/5.0):** El estudiante no entregó una respuesta para la pregunta.